

Phase 1 Status Report

Multi-Pipeline ETL and Reporting Framework for Web Server Log Analytics

DAS 839 – NoSQL Systems End Semester Project

IMT2022021 Rutul Patel
IMT2022049 Tanish Pathania
IMT202076 Mohit Naik
IMT202206 Ananthakrishna K

April 28, 2026

Abstract

This report outlines the Phase 1 implementation of a prototype multi-pipeline Extract, Transform, Load (ETL) and reporting framework. Designed to process NASA HTTP web server logs, the system orchestrates analytics workloads across four distinct Big Data execution paradigms: Apache MapReduce, Apache Pig, Apache Hive, and MongoDB. The system successfully extracts raw logs, parses them into a structured format, executes three mandatory aggregation queries, loads the final results into a PostgreSQL relational database, and renders a comparative telemetry dashboard.

1 Introduction and Objectives

The primary objective of this project is to implement and evaluate different data processing paradigms for semi-structured log analytics. To ensure a fair comparison regarding implementation style, execution time, and batching behavior, a unified orchestration framework was developed in Java. This framework guarantees that the same logical dataset and semantic query definitions are applied consistently, regardless of the underlying execution engine.

2 System Architecture

The project relies on a deeply containerized microservices architecture managed via Docker Compose. The environment consists of modular configurations (`docker-compose.*.yaml`) that dynamically spin up the required database nodes and application containers.

2.1 Orchestration & Ingestion (Main.java)

A central orchestrator coordinates the entire ETL flow. It performs the following duties:

- **Configuration & Bootstrapping:** Reads environment variables (e.g., `PIPELINE_NUMBER`, `BATCH_SIZE`) and instantiates the selected pipeline strategy using the Strategy Design Pattern via the `Pipeline` interface.
- **Extraction:** Streams the `NASA_access_log_sample` dataset.
- **Parsing:** Utilizes `LogParser.java` with Regular Expressions to extract 9 specific fields (e.g., Host, Timestamp, Method, Resource, Status Code, Bytes). Malformed records are tracked and dropped.

- **Batching:** Accumulates parsed `NasaLogRecord` objects into memory and flushes them to the chosen engine via `loadBatch()` once the batch size threshold is reached.

3 Pipeline Implementations (The Transform Phase)

Once data is staged, the framework triggers the `executeQueries()` method on the selected interface, delegating the analytical heavy lifting to the Big Data engines.

3.1 MongoDB Pipeline

For the NoSQL Document model, the batch is inserted directly into a collection as BSON documents. Analytics are executed using MongoDB's native Aggregation Framework.

- Uses `$group` and `Accumulators` to calculate requested metrics.
- Leverages `$cond` inside `$addToSet` alongside `$size` to accurately calculate `COUNT(DISTINCT host)` for error-specific telemetry.

3.2 Apache Pig Pipeline

For the scripting paradigm, data is written to intermediate `.tsv` files. Pig Latin scripts are generated dynamically and executed via the `PigServer` local engine.

- Uses `PigStorage` to load TSV data.
- Utilizes nested `FOREACH` and `DISTINCT` filters to calculate aggregation metrics and distinct hosts.

3.3 Apache Hive Pipeline

For the SQL-on-Hadoop paradigm, Hive processes the staged `.tsv` data as an external table.

- A JDBC connection is established to `HiveServer2`.
- Standard `HiveQL` is utilized to perform `GROUP BY` and `COUNT(DISTINCT ...)` operations natively.

3.4 Apache MapReduce Pipeline

For the low-level processing paradigm, custom Mapper and Reducer classes were written.

- Implements custom `KeyWritable` and `ValueWritable` classes to manage composite grouping keys and multiple accumulators.
- Resolves distinct host counts by emitting host strings during the Map phase and aggregating them via `HashSet.size()` during the Reduce phase.

4 Load and Reporting Phase

Instead of dumping intermediate data, each pipeline's final aggregation result is loaded directly into a PostgreSQL relational database.

4.1 Database Schema

The PostgreSQL instance is initialized via `nasa_analytics.sql` with the following entities:

- **Metadata:** `execution_metadata`, `query_metrics`
- **Target Tables:** `q1_daily_traffic`, `q2_top_resources`, `q3_hourly_errors`

4.2 Reporting Module

The `ReportingModule.java` executes a series of `SELECT` queries against PostgreSQL to generate a formatted CLI dashboard. It provides the end-user with execution telemetry (Run ID, Total Runtime, Average Batch Size) and the summarized results of the analytics queries.

5 Current Status and Conclusion

Phase 1 of the project is **complete and operational**. All four pipelines have been thoroughly tested on the 10,000-record sample dataset.

Validation Metrics (Sample Run):

- **MongoDB:** 628 ms
- **Apache Pig:** 5,681 ms
- **Apache Hive:** 7,382 ms
- **MapReduce:** 9,940 ms

All pipelines successfully parse the logs, discard 0 malformed records (on the clean sample), and populate the relational database with identical results. The system is ready for Phase 2 scaling and final video demonstration.